Object Oriented Programming with C++

ASSIGNMENT 1

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

Assignment Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System (Total Marks - 50)

Nature of Assignment : Individual
Duration : 1 Week
Submission Date : 10/11/2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: https://github.com/192521288/SLOT-C-OOP.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

TASK 1 – Understand the Problem and Identify the Objects (4 Marks)

Objects, data and behaviour

Object	Data	Behaviour / reason
Passenger	ID, name, passenger category	Determines eligibility for student or senior concession.
SmartCard	Card ID, private balance in paise	canTravel(), deduct(), recharge(); protects balance rules.
Journey	Origin, farthest zone, time	Validates a trip and supplies fare inputs.
Zone	Inner, Middle, Outer	Finite set of zone values; later Airport is added.
FareTable	Base fares, surcharge, reserve balance	Centralises fare policy so revisions are made in one place.

Procedural versus OOP

A procedural program stores balance, zone, time and fare in separate variables and uses free functions such as calculateFare(), canTravel() and deductFare(). Any function with access to the balance could accidentally change it. In OOP, related data and operations are grouped into classes. SmartCard owns its balance, FareTable owns pricing rules and Journey represents one trip.

OOP principles

- Encapsulation: SmartCard keeps balance private and changes it only through checked member functions.
- Abstraction: the gate calls fareFor() and canTravel() without needing to know internal arithmetic details.
- Inheritance: StudentPassenger and SeniorPassenger can derive from Passenger and implement different concession policies.
- Polymorphism: a virtual concessionRate() called through a Passenger reference selects the derived implementation at run time.

Journey invariants

- Zone and time must be valid.
- Fare is non-negative and represented in integer paise.
- Peak time is (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20).
- After deduction, balance must be at least 5000 paise.
- A denied journey must not alter the card balance.
- Fare constants must be defined centrally.

Alternative concept and complexity

A procedural solution can use a balance variable and free functions. Both procedural and OOP fare calculations take O(1) time per journey and use O(1) memory per card. OOP does not improve the asymptotic complexity; it improves ownership, encapsulation and maintainability. A centralised rule is

easy to change in either design, but duplicated procedural rules require more edits and are more likely to become inconsistent.

Class relationship sketch: Passenger is the base class of StudentPassenger and SeniorPassenger. Gate uses Journey, FareTable and SmartCard. Journey refers to Zone. SmartCard encapsulates balance.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly (3 Marks)

Token type	Meaning	Examples
Keywords	Reserved language words	class, enum, constexpr, if, switch, return, bool
Identifiers	Programmer-defined names	SmartCard, balancePaise, fareFor
Constants/literals	Fixed values	5000, 20, 1000, true, Zone::Inner
Operators	Perform operations	+, -, *, /, %, ==, >=, &&, ||, -=
Punctuators	Structure statements	{ }, ( ), [ ], ;, ,, ::

Constant choices

#define performs preprocessor text substitution and has no normal C++ type or scope. const declares an object that cannot be modified after initialisation. constexpr requires a compile-time constant expression where applicable and is preferred for fixed fare policy.

```
constexpr long long INNER_FARE_PAISE = 1000;\nconstexpr long long MIDDLE_FARE_PAISE =
2000;\nconstexpr long long OUTER_FARE_PAISE = 3000;\nconstexpr int PEAK_SURCHARGE_PERCENT =
20;\nconstexpr long long MIN_BALANCE_PAISE = 5000;
```

enum class Zone { Inner, Middle, Outer }; is safer than an unscoped enum because enumerators are scoped and do not implicitly convert to int. Use PascalCase for class names, camelCase for variables/functions and uppercase underscore names for constants. Identifiers cannot start with digits, contain spaces or be reserved keywords.

Alternative concept and complexity

Using #define macros instead of constexpr and enum class removes type checking and ordinary scope protection. A macro occupies no storage; a constexpr constant generally needs no storage when used as a compile-time value. Running time is normally unchanged after compilation. constexpr plus enum class is safer and easier to maintain.

TASK 3 – Choose Data Types and Check Type Compatibility (4 Marks)

Value	Recommended type	Reason / typical size
Fare and balance	long long paise	Exact integer money; commonly 8 bytes.
Hour / minute	int	Small whole numbers; commonly 4 bytes.
Zone	enum class Zone	Named finite values; commonly 4 bytes underlying.
Journey count	long long / uint64_t	Large counts; commonly 8 bytes.
Peak / eligibility	bool	True/false; commonly 1 byte.
Character	char	Single character; commonly 1 byte.
Ratio / measurement	double	Floating-point measurement; commonly 8 bytes.

Why integer paise?

Rs 10.25 is represented as 1025 paise. Integer addition and multiplication remain exact until overflow. Many decimal fractions cannot be represented exactly in binary floating point, so double rupees can accumulate rounding differences.

Type compatibility

```
int count = 3;\ndouble rate = 2.5;\nbool valid = true;\ndouble total = count * rate + valid;
```

count is converted to double for multiplication. true is promoted to integer 1 and then converted to double for addition. The result is 8.5. sizeof(type) and limits from <climits> should be printed on the actual g++ system because sizes are implementation-dependent.

Network collection and overflow

Journeys	At Rs 10 each	Total paise	Fits signed 32-bit int?
20,000	Rs 200,000	20,000,000	Yes
500,000	Rs 5,000,000	500,000,000	Yes
3,000,000	Rs 30,000,000	3,000,000,000	No

A typical signed 32-bit int has maximum 2,147,483,647. Therefore, 3,000,000,000 paise does not fit. A signed long long is sufficient.

Alternative concept and complexity

Using double rupees still gives O(1) time per addition and typically 8 bytes per value, the same typical size as long long. However, decimal fractions can be approximate. Test a fare such as Rs 0.10 repeated many times against integer paise. The Rs 10 example is exactly representable as a whole number in double and does not demonstrate a rounding error; do not claim a measured error until you run the fractional-fare experiment.

TASK 4 – Control Implicit Conversions (4 Marks)

- Integer division: 7 / 2 produces integer 3; assigning it to double gives 3.0 after the fraction is lost.
- Explicit conversion: static_cast<double>(7) / 2 produces 3.5.
- char promotion: char participates in arithmetic after promotion to int.
- int-to-bool: zero becomes false; any non-zero integer becomes true.
- Signed/unsigned comparison: -1 may convert to a large unsigned value, so comparing signed -1 with unsigned 1 can produce an unexpected result.

Exact output of the supplied program

```
3 3.5\nC 68\n1 true\ns >= u
```

Line 1: a / b is integer division, so c is 3.0. static_cast<double>(a) makes the second division floating point, producing 3.5. Line 2: ch is A + 2, hence C; ch + 1 promotes char to int and prints 68. Line 3: 5 converts to true; default bool output is 1, and boolalpha prints true. Line 4: s = -1 is converted to unsigned for the comparison and becomes a large unsigned value, so s < u is false.

Safer style and alternative

```
int a{7}, b{2};\ndouble d = static_cast<double>(a) / b;\n// int bad{3.7}; // compile-time
narrowing error
```

Brace initialisation rejects narrowing conversions such as int x{3.7}. Explicit static_cast documents intentional conversion. Making conversions explicit normally does not change time or memory complexity; it makes intent visible and catches some mistakes at compile time. Explicit casts can still be logically unsafe, so validation remains necessary.

TASK 5 – Write Correct Expressions with the Right Precedence (4 Marks)

Peak, surcharge and concession expressions

```
bool peak = (hour >= 8 && hour < 10) ||\n            (hour >= 17 && hour < 20);\nlong long fare = baseFarePaise;\nif (peak) fare += fare * 20 / 100;\nif (eligibleForConcession)\n    fare = fare * (100 - concessionPercent) / 100;
```

The concession percentage is not specified in the assignment. It must be supplied by the official fare policy. The shown order applies concession after peak surcharge; change it if the metro policy defines a different order.

Precedence and associativity

- Multiplication, division and remainder bind more tightly than addition; relational operators bind before logical AND; && binds before ||.
- The parenthesised peak expression is clearest. The unparenthesised expression is equivalent because && has higher precedence than ||.
- ?: selects one of two expressions. Compound assignment fare += x adds x to fare.
- x++ yields the old value then increments; ++x increments first. Avoid i = i++ because it has undefined behaviour.
- Integer division truncates toward zero: -10 / 3 is -3. The remainder is -1: -10 % 3.

Exact output of supplied operator program

```
11 16\n9 6 4\n6 4\n3 1 -3 -1\n18
```

Line 1: multiplication precedes addition; parentheses force the second addition first. Line 2: x++ contributes 5 then x becomes 6; ++y changes y to 4 and contributes 4, so z = 9. Line 3: the conditional operator selects x = 6; (x % 4) * 2 = 4. Line 4 demonstrates integer division and remainder for positive and negative values. Line 5 sums i = 0, 3, 6, 9, giving 18.

Alternative lookup table

A bool peakHour[24] table can make a peak check a single O(1) array lookup, compared with a constant number of comparisons in the logical expression. It requires 24 bool elements (often 24 bytes, but sizeof(bool) is implementation-defined) and careful initialisation and bounds checks.

TASK 6 – Design the Control Structures (4 Marks)

- Use if–else for validation, surcharge and balance checks; switch is suitable for selecting one of a finite set of zones; ?: is suitable only for short value selection.
- A range-based for loop suits a known collection; a counted for loop suits a known count; while suits input until a sentinel or end-of-file.
- continue skips an invalid journey; break exits a loop or switch. A switch default branch handles an unmatched value.

Pseudocode

```
ALGORITHM ProcessJourneys\nSET balancePaise to initial balance\nFOR each journey\n  IF zone or
time is invalid THEN print error; CONTINUE\n  SELECT zone: Inner=1000, Middle=2000, Outer=3000\n
OTHERWISE: print invalid zone; CONTINUE\n  peak <- (hour>=8 AND hour<10) OR (hour>=17 AND
hour<20)\n  fare <- base fare\n  IF peak THEN fare <- fare + fare*20/100\n  IF balancePaise - fare
>= 5000 THEN\n    balancePaise <- balancePaise - fare; print accepted\n  ELSE print denied\nEND
FOR
```

Completed TODO program

```
#include <iomanip>
#include <iostream>
enum class Zone { Inner, Middle, Outer };
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;


long long fare(Zone z, bool peak) {
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
        default: return -1;
    }
    if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;
    return base;
}
bool canTravel(long long balance, long long f) {
    return f >= 0 && balance >= f &&
           balance - f >= MIN_BALANCE_PAISE;
}
int main() {
    long long balance = 10000;
    struct Trip { Zone zone; bool peak; };
    Trip trips[] = {{Zone::Inner,false},
                    {Zone::Outer,true},
                    {Zone::Middle,false}};
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            std::cout << "Fare Rs " << f/100 << '.'
                      << std::setw(2) << std::setfill('0')
                      << f%100 << std::setfill(' ');
        } else std::cout << "Denied (low balance)";
```

```
        std::cout << " balance Rs " << balance/100 << '\\n';
    }
}
```

Expected output:

```
Fare Rs 10.00 balance Rs 90\nFare Rs 36.00 balance Rs 54\nDenied (low balance) balance Rs 54
```

At exactly 10:00 AM and 8:00 PM the journey is off-peak because the upper bounds are exclusive.

Alternative array lookup and complexity

An array indexed by a validated zone index and a switch both provide O(1) selection. An array of three long long values typically occupies 24 bytes. A switch may compile to comparisons or a jump table. Adding Airport requires extending the enum and array or adding another switch case.

TASK 7 – From Procedural to Object-Oriented (4 Marks)

Procedural version

```
long long calculateFare(int zone, bool peak) {
    long long f = zone==0 ? 1000 : zone==1 ? 2000 :
                  zone==2 ? 3000 : -1;
    if (f < 0) return -1;
    if (peak) f += f * 20 / 100;
    return f;
}
bool canTravel(long long balance, long long f) {
    return f >= 0 && balance >= f && balance-f >= 5000;
}
bool deduct(long long& balance, long long f) {
    if (!canTravel(balance,f)) return false;
    balance -= f; return true;
}
```

Object-oriented SmartCard version

```
class SmartCard {
private:
    long long balancePaise;
    static constexpr long long MIN_BALANCE = 5000;
public:
    explicit SmartCard(long long initial) : balancePaise(initial) {}
    bool canTravel(long long f) const {
        return f >= 0 && balancePaise >= f &&
               balancePaise-f >= MIN_BALANCE;
    }
    bool deduct(long long f) {
        if (!canTravel(f)) return false;
        balancePaise -= f; return true;
    }
    long long balance() const { return balancePaise; }
};
```

Both designs can perform an O(1) deduction and use O(1) storage per card. In the procedural design, callers must consistently use the validation function. In the OOP design, private balance prevents arbitrary external modification. A public balance field would allow any component to set a negative value or bypass the Rs 50 reserve rule. Time and space do not change, but correctness protection is lost.

TASK 8 – Read and Print Data Safely (4 Marks)

Safe input with cin

```
#include <iostream>
#include <limits>
int main() {
    int hour{};
    while (true) {
        std::cout << "Enter hour (0-23): ";
        if (std::cin >> hour && hour >= 0 && hour <= 23) break;
        if (std::cin.eof()) return 1;
        std::cout << "Invalid input. Try again.\\n";
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\\n');
    }
    std::cout << "Accepted hour: " << hour << '\\n';
}
```

If letters are entered, cin.fail() becomes true. cin.clear() resets stream error flags; cin.ignore(max, newline) discards the invalid input line. Always check the numeric range after extraction.

Currency output

```
#include <iomanip>
#include <iostream>
void printMoney(long long p) {
    std::cout << "Rs " << p/100 << '.'
              << std::setw(2) << std::setfill('0')
              << p%100 << std::setfill(' ');
}
```

setw sets the next field width; setfill sets its padding character; fixed and setprecision format floating-point values. For integer paise, quotient and remainder by 100 print exact rupees and two paise digits.

Alternative getline and stoi

getline reads a complete line; stoi converts it and may throw invalid_argument or out_of_range. A try/catch handles these errors, and the consumed-character count should be checked to reject trailing junk. Reading and converting a line of length L takes O(L) time and O(L) buffer space. cin extraction is convenient for tokens; getline gives control over the complete line and its validation.

TASK 9 – Implement and Test the Fare Logic (4 Marks)

Test cases and expected results

Test	Input	Expected result
1	Inner, 11:00, balance Rs 100	Fare Rs 10; accepted; balance Rs 90
2	Middle, 08:00, balance Rs 100	Fare Rs 24; peak boundary included
3	Outer, 10:00, balance Rs 100	Fare Rs 30; off-peak boundary
4	Outer, 17:00, balance Rs 100	Fare Rs 36; peak boundary included
5	Outer, 20:00, balance Rs 100	Fare Rs 30; off-peak boundary
6	Inner, balance Rs 60	Accepted; exactly Rs 50 remains
7	Outer peak, balance Rs 80	Denied; would leave Rs 44
8	Unknown zone	Reject; balance unchanged
9	Negative recharge	Reject; balance unchanged
10	Letters entered for hour	Report invalid input; recover and reprompt

Five journey hand calculation

Journey	Zone/time	Fare	Sequential balance
1	Inner, 11:00	Rs 10	Rs 90
2	Middle, 08:30	Rs 24	Rs 66
3	Outer, 12:00	Rs 30	Denied; Rs 66 remains
4	Inner, 17:30	Rs 12	Accepted; Rs 54 remains
5	Middle, 20:00	Rs 20	Denied; Rs 54 remains

The third and fifth journeys are denied because they would leave less than Rs 50. The fourth journey is accepted. The balance is unchanged by denied journeys. Concession tests require the official concession percentage, which is not specified in the assignment.

Alternative test-table design

Store input cases and expected fares in an array/vector and loop over t cases. Runtime is O(t), and table storage is O(t). A new boundary case can be added as a row, and each result can be reported as PASS or FAIL. Fill the actual-result column after executing the program; attach real screenshots.

TASK 10 – Compare Expectations with Execution (4 Marks)

For a controlled scale comparison, assume every journey costs exactly Rs 10 (1000 paise).

Journeys	Integer total paise	Rupee total	32-bit signed int?
20,000	20,000,000	Rs 200,000	Fits
500,000	500,000,000	Rs 5,000,000	Fits
3,000,000	3,000,000,000	Rs 30,000,000	Does not fit

```
#include <iomanip>
#include <iostream>
int main() {
    const long long counts[] = {20000, 500000, 3000000};
    for (long long n : counts) {
        long long paise = n * 1000LL;
        double rupees = static_cast<double>(n) * 10.0;
        std::cout << n << " journeys: " << paise
                  << " paise; Rs " << std::fixed
                  << std::setprecision(2) << rupees << '\\n';
    }
}
```

Expected totals: 20,000 journeys = 20,000,000 paise; 500,000 = 500,000,000 paise; 3,000,000 = 3,000,000,000 paise. In this specific Rs 10 experiment, double rupees can represent the whole-number totals exactly. To investigate floating-point rounding, repeat a fractional fare such as Rs 0.10 and compare the double result with integer paise.

Warnings and complexity

Compile with g++ -std=c++17 -Wall -Wextra -Wconversion -pedantic file.cpp -o program. -Wall and -Wextra enable warning groups; -Wconversion warns about potentially value-changing conversions; -pedantic diagnoses non-standard extensions. Record only warnings actually produced by your compiler. int paise overflows at 3,000,000,000; signed overflow is undefined behaviour. long long paise safely holds this total. double has O(1) addition and commonly uses 8 bytes but decimal fractions can be approximate. Each addition is O(1); typical int storage is 4 bytes and long long/double 8 bytes.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

Recommended design

- Use long long integer paise for fares, balances and collection totals.
- Use enum class Zone and constexpr constants for zone fares, surcharge and minimum balance.
- Use SmartCard with private balance and checked recharge(), canTravel() and deduct() operations.
- Keep fare policy in a central FareTable/FarePolicy component and represent each trip as a Journey.
- Use explicit half-open peak intervals: 08:00 <= time < 10:00 and 17:00 <= time < 20:00.

Revision, limitations and scale

For a fare revision, update the central fare table, update relevant tests, compile, run regression and boundary tests, then deploy. Limitations: concession percentages and their order are unspecified; the model does not include distance-based fares, transfers, refunds, fare caps, authentication or persistent transaction storage. For N journeys, total processing time is O(N); one fare calculation and card check are O(1). long long is sufficient for the illustrative whole-network total.

Alternative configuration-file design

Loading z fare values from a configuration file takes O(z) startup time and O(z) storage. It avoids recompiling for every fare revision, but the application must validate missing/negative/invalid values and preserve a last-known-good configuration. A compiled constexpr table has negligible startup cost but requires rebuild and redeployment after policy changes.

TASK 12 – Re-evaluate and Adapt the Design – Day-4 Challenge (8 Marks)

New requirements: Airport zone at Rs 60; weekend discount of 10% that does not combine with peak surcharge; monthly pass for unlimited journeys in two selected zones.

A. Impact analysis

- Add Airport to enum class Zone and add 6000 paise to the fare table.
- Change the fare calculation to distinguish weekend and weekday peak. Weekend discount and peak surcharge must be mutually exclusive.
- Add Airport handling to the switch or extend a zone-indexed fare table.
- Keep SmartCard for stored value and add a separate MonthlyPass class with two covered zones and validity dates.

B. Evaluate the original design

The switch design can be extended by adding a new enumerator and case, but repeated special cases make it longer. A validated fare table is easier to extend. SmartCard should remain responsible for stored value; monthly-pass eligibility belongs in a separate class.

Scenario	Calculation	Fare
Airport weekday off-peak	6000 paise	Rs 60.00
Airport weekday peak	6000 + 20%	Rs 72.00
Airport weekend	6000 - 10%; no peak surcharge	Rs 54.00
Covered zone with valid pass	Pass covers trip and date	Rs 0.00

C. Modified implementation

```
#include <array>
#include <cstddef>
#include <iostream>
enum class Zone { Inner, Middle, Outer, Airport };
constexpr std::array<long long,4> BASE = {1000,2000,3000,6000};

long long calculateFare(Zone z, int hour, bool weekend) {
    int i = static_cast<int>(z);
    if (i < 0 || i >= static_cast<int>(BASE.size())) return -1;
    long long f = BASE[static_cast<std::size_t>(i)];
    if (weekend) f -= f * 10 / 100;
    else if ((hour >= 8 && hour < 10) ||
             (hour >= 17 && hour < 20))
        f += f * 20 / 100;
    return f;
}
class MonthlyPass {
    Zone a, b;
    int firstDay, lastDay;
public:
    MonthlyPass(Zone x, Zone y, int start, int end)
        : a(x), b(y), firstDay(start), lastDay(end) {}
    bool covers(Zone z, int day) const {
        return (z == a || z == b) &&
               day >= firstDay && day <= lastDay;
```

```
    }
};
class SmartCard {
    long long balancePaise;
    static constexpr long long MIN_BALANCE = 5000;
public:
    explicit SmartCard(long long b) : balancePaise(b) {}
    bool deduct(long long f) {
        if (f < 0 || balancePaise < f ||
            balancePaise - f < MIN_BALANCE) return false;
        balancePaise -= f; return true;
    }
    long long balance() const { return balancePaise; }
};
int main() {
    MonthlyPass pass(Zone::Inner, Zone::Middle, 1, 30);
    SmartCard card(20000);
    Zone trip = Zone::Airport;
    int day = 12, hour = 18;
    bool weekend = true;
    if (pass.covers(trip, day))
        std::cout << "Covered by pass; fare Rs 0.00\\n";
    else {
        long long f = calculateFare(trip, hour, weekend);
        if (f >= 0 && card.deduct(f))
            std::cout << "Fare paise: " << f
                      << "; balance paise: " << card.balance() << '\\n';
        else std::cout << "Journey denied\\n";
    }
}
```

Expected sample output: the pass covers Inner and Middle only, so Airport is not covered. Weekend discount applies instead of peak surcharge. Airport fare = 6000 - 600 = 5400 paise; balance becomes 20000 - 5400 = 14600 paise.

```
Fare paise: 5400; balance paise: 14600
```

The sample pass uses an illustrative integer day range. A production pass should use a proper date/time type and validate calendar-month boundaries, expiry and renewal.

D. Evidence-based defence

Decision: MODIFY the original design. Retain integer-paise arithmetic and SmartCard encapsulation because they already provide exact currency handling and protect the minimum balance. Extend the zone model and fare table, separate weekend and peak branches so they cannot combine, and introduce MonthlyPass as a separate class. This avoids mixing stored-value and pass rules. Compile and execute weekday peak/off-peak, weekend, Airport, pass-covered and uncovered-zone tests before claiming measured evidence.